

```

Attempting to solve in at most 10 moves...
Attempting to solve in at most 11 moves...
Attempting to solve in at most 12 moves...
--snip--
 1  2  3  4
 5  6  7  8
 9 10 11 __
13 14 15 12

Move up

 1  2  3  4
 5  6  7  8
 9 10 11 12
13 14 15 __

Solved in 18 moves:
left, down, right, down, left, up, right, up, left, left, down,
right, right, up, left, left, left, up
Run in 39.519 seconds.

```

Note that when JavaScript runs in a browser, the code must complete before it displays any output. Until then, it may appear to have frozen, and your browser might ask if you'd like to prematurely stop it. You can ignore this warning and let the program keep working until it has solved the puzzle.

The program's recursive `attemptMove()` function solves sliding-tile puzzles by trying every possible combination of slides. The function is given a move to try. If this solves the puzzle, the function returns a Boolean `True` value. Otherwise, it calls `attemptMove()` with all the other possible moves it can make and returns a Boolean `False` value if none of them find a solution before exceeding the maximum number of moves. We'll explore this function in more detail later.

The data structure we use to represent a sliding-tile board is a list (in Python) or array (in JavaScript) of integers, with `0` representing the blank space. In our program, this data structure is often stored in a variable named `board`. The values at `board[y * SIZE + x]` match the tile at the `x`, `y` coordinates on the board, as depicted in [Figure 12-5](#). For example, if the `SIZE` constant is 4, the value at the `x`, `y` coordinates 3, 1 can be found at `board[1 * 4 + 3]`.